# EXPERIMENT: 08

**Name:-** Ayush Tiwari
**UID:-** 23BCS11366
**Branch:-** BE-CSE
**Section:-** KRG 3B
**Semester:-** 6th
**Date :-** 08-04-2026
**Subject Name:-** System Design
**Subject Code:-** 23CSH-314

## 1.Aim:-

To design a high-level architecture for a scalable stock trading platform (similar to Zerodha, Groww, and Upstox) that supports real-time stock trading, order management, and portfolio tracking with high availability and low latency.

## 2.Objectives:-

The objectives of this experiment are:

1. To understand the architecture of a real-time stock trading system.
2. To design APIs for user authentication, KYC verification, market data, and order execution.
3. To analyze functional and non-functional requirements of a trading platform.
4. To study how real-time data streaming and event-driven systems work.
5. To ensure consistency in financial transactions while maintaining scalability.
6. To understand how portfolio and trade management systems operate.

## 3.Procedure:-

- Studied real-world stock trading platforms such as Zerodha, Groww, and Upstox to understand system behavior.
- Identified key system entities including Users, Stocks, Orders, Trades, Portfolio, and Watchlist.
- Analyzed the complete workflow from user registration → KYC → viewing stocks → placing orders → trade execution.
- Gathered functional requirements such as real-time stock updates, order placement, and portfolio tracking.
- Analyzed non-functional requirements including low latency, high availability, fault tolerance, and scalability.
- Designed RESTful APIs for user management, market data, portfolio, and watchlist features.
- Studied WebSocket-based real-time updates for stock prices.
- Understood event-driven architecture using message queues (Kafka) for trade execution and notifications.

• Evaluated trade-offs between strong consistency (orders, trades) and eventual consistency (analytics, notifications).

## 4. Functional Requirements:-

Users should be able to register, log in, and complete KYC verification.
• Users should be able to view real-time stock prices and historical data.
• The system should support placing, modifying, and canceling orders.
• Orders should provide accurate real-time status updates.
• Users should be able to maintain a watchlist with live updates.
• The system should provide a dashboard to track holdings, trade history, and P&L.
• Portfolio management should reflect executed trades instantly.

## 5.Non Functional Requirements:-

• The system must support millions of users simultaneously.
• Real-time stock updates should have very low latency (<100 ms).
• High availability (99.9%+) must be ensured for trading operations.
• Strong consistency is required for order execution and trades.
• The system should be fault tolerant and resilient to failures.
• Scalability should be achieved using microservices architecture.
• Secure communication and authentication (JWT, HTTPS) must be implemented.

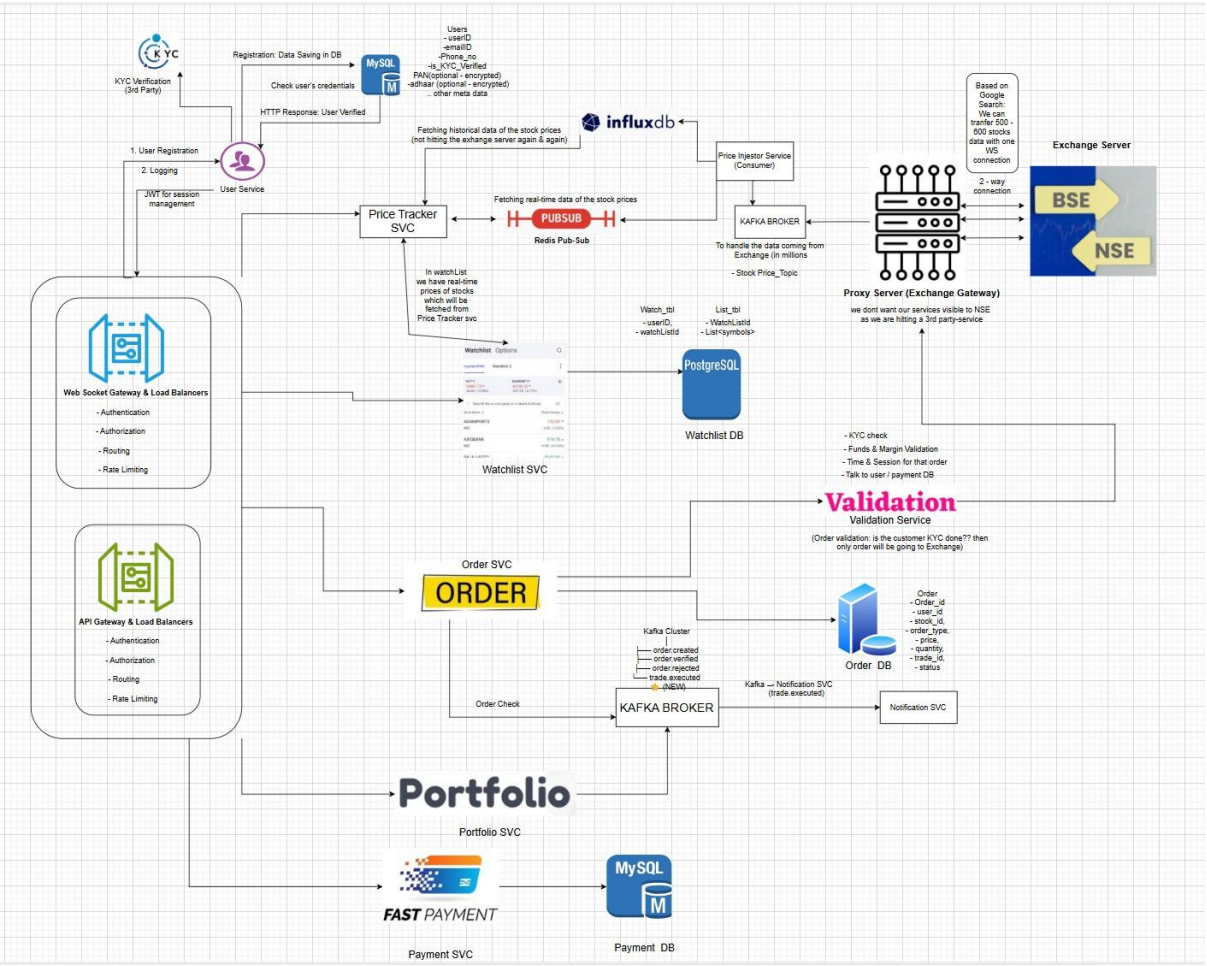
## 6.Core Entities:-

• User
• Stock
• Order
• Trade
• Portfolio
• Watchlist

## 7.API Design:-

**User APIs:**
• POST /auth/signup
• POST /auth/login
• POST /auth/verify-KYC **Market Data APIs:**
• WS /api/v1/stocks → Real-time stock updates
• GET /api/v1/stocks → Get all stocks
• GET /api/v1/stocks/{stock_symbol_id} → Stock details • GET /api/v1/stocks/{stock_symbol_id}/history → Historical data **Order APIs:**
• POST /api/v1/orders → Place order
• PUT /api/v1/orders/{order_id} → Modify order
• DELETE /api/v1/orders/{order_id} → Cancel order

• GET /api/v1/orders → Get user orders

**Portfolio APIs:**
• GET /api/v1/portfolio
• GET /api/v1/portfolio/holdings
• GET /api/v1/portfolio/positions
• GET /api/v1/portfolio/performance (Optional) **Watchlist APIs:**
• GET /api/v1/watchlists
• POST /api/v1/watchlists/{watchlist_id}/symbols
• DELETE /api/v1/watchlists/{watchlist_id}/symbols/{symbol}

## 8.HLD:-